Flow Matching Algorithm: Considerations about practical implementation

Marco Cococcioni

Dept of Information Engineering

4 June 2025

Practical Considerations for Flow Matching

- In this slide, we present some considerations about the practical implementation of the algorithm.
- Some tools to optimize the algorithm and make the best use of the data (temporal structure...).

Equivariant Architectures

- **Equivariance**: a model M is equivariant to a transformation f if:

$$M \circ f(x) = f \circ M(x)$$

- **Why use them?**
  - Many data domains exhibit inherent symmetries (rotations, reflections, permutations).
  - Encoding these symmetries improves generalization and reduces data requirements.
- **Examples**:
  - Rotation-equivariant digit recognition
  - Permutation-equivariant models for graphs (node order doesn't matter)
  - Reflection-equivariant models in chemistry or physics

Why Equivariance Helps

- **Without equivariance**, a standard neural network must empirically learn that certain inputs are equivalent (e.g., rotated or translated versions of the same image).
- This requires:
  - More training examples covering each transformation,
  - Larger model capacity to memorize redundant patterns,
  - Slower learning and weaker generalization.
- **Example: CNNs** are translation-equivariant: shifting an image shifts the feature map.

Stochastic Flow Matching: Core Idea

- **Goal:** Learn a time-dependent vector field $u_t^\theta(x)$ that transports samples from a simple distribution p_0 (e.g., Gaussian) to a complex data distribution p_1, using stochastic dynamics.
- We define a stochastic process (SDE):

$$dx_t = d\phi_t(x_0) = u_t^\theta(x_t)\,dt + \sigma(t)\,dB_t$$

  - $u_t^\theta(x)$: learnable drift field (neural network)
  - $\sigma(t)$: time-dependent noise level
  - B_t: Brownian motion (random perturbation)
- **Interpretation:**
  - Starting from $x_0 \sim p_0$, samples follow noisy paths toward p_1
  - The noise ensures diversity and smoothness in paths
- **Why stochastic?** It improves robustness, mode coverage, and training stability.

What the Model Learns

- **Objective:** Learn $u_t^\theta(x) \approx \mathbb{E}[\dot{x}_t = \frac{d}{dt}\phi_t(x_0) \mid x_t = \phi_t(x_0)]$
  - The drift field predicts the expected direction of movement given the current position x_t
  - This is estimated from stochastic paths between $x_0 \sim p_0$ and $x_1 \sim p_1$
- **Training procedure:**
  1. Sample $x_0 \sim p_0$, $x_1 \sim p_1$ in our dataset
  2. Interpolate them via a noisy path (e.g., straight line + Brownian noise)
  3. At random time $t \in [0, 1]$, sample a point x_t on the path
  4. Compute or approximate the conditional velocity $\dot{x}_t = \frac{d}{dt}\phi_t(x_0)$
  5. Minimize the squared error:

$$\mathcal{L}_{\text{SFM}}(\theta) = \mathbb{E}_{t,x_t}\left[\left\| u_t^\theta(x_t) - \dot{x}_t \right\|^2\right]$$

- **Outcome:** The learned drift field guides a stochastic sampler to generate realistic data from noise.

Path Regularization Techniques

- When training flow or velocity-based generative models, controlling the smoothness and stability of learned velocity fields is crucial.
- Path regularization helps enforce desirable properties on these vector fields along the trajectories.
- Two popular techniques:
  - **Sobolev** norms on velocity fields
  - **Spectral** normalization for stability

Sobolev Norms on Velocity Fields

- Sobolev norms measure not only the magnitude but also the smoothness of a function.
- Applying Sobolev norms on velocity fields encourages smooth vector fields by penalizing high derivatives.
- Formally, for a velocity field $u(x)$, the Sobolev norm of order k involves derivatives up to order k:

$$\|u\|_{H^k} = \left(\sum_{|\alpha| \leq k} \|\partial^\alpha u\|_2^2 \right)^{1/2}$$

- This regularization leads to smoother flows and more stable training dynamics.

Spectral Normalization and the Spectrum

- Each neural network layer is represented by a weight matrix W.
- The **spectral norm** $\sigma_{\max}(W)$ is the largest singular value[1] of W, measuring its maximum amplification of inputs.
- Large spectral norms can cause unstable activations and irregular velocity fields.
- **Spectral Normalization** rescales weights to control this:

$$W_{SN} = \frac{W}{\sigma_{\max}(W)}$$

  ensuring a Lipschitz constant ≤ 1.
- This stabilizes training by preventing abrupt changes and exploding gradients in the velocity field.

[1] Square root of the eigenvalues of $W^T W$.

Implicit and Semi-Implicit Integration

- **Explicit Euler** (used in Slide 10):

$$x_{t+\Delta t} = x_t + \Delta t \cdot u_t(x_t)$$

  Simple and efficient, but unstable for stiff velocity fields unless Δt is very small.

- **Implicit Euler** improves stability:

$$x_{t+\Delta t} = x_t + \Delta t \cdot u_t(x_{t+\Delta t})$$

  Since $x_{t+\Delta t}$ appears on both sides, we must solve a nonlinear system at each step — expensive.

- **Semi-Implicit Euler**:

$$u_t(x_{t+\Delta t}) \approx u_t(x_t) + \frac{\partial u_t(x)}{\partial x}\Big|_{x=x_t} \cdot (x_{t+\Delta t} - x_t)$$

  Plugged into the update, this yields a linear system — more stable than explicit, cheaper than full implicit.

Multi-Scale Flow Matching

- Instead of directly learning a high-resolution flow from scratch, we process the data at multiple spatial scales by progressively downsampling the inputs.
- Coarse, low-resolution scales are used to capture global structure and large motion patterns.
- Higher-resolution scales refine the flow by adding increasingly fine-grained local details.
- This coarse-to-fine refinement strategy improves:
  - training stability,
  - convergence speed,
  - and generation quality.
- The flow at each scale acts as an initial estimate or scaffold for the next level:

$$\text{Flow}^{(s+1)} = \text{Refine}(\text{Flow}^{(s)})$$

Refinement Step in Multi-Scale Flow Matching

- At each scale s, we learn a velocity field $u_t^{\theta,(s)}(x^{(s)})$.
- To move to the next (finer) scale $s+1$, we first upsample this velocity field:

$$\tilde{u}_\theta^{(s+1)} = \text{Upsample}(u_\theta^{(s)})$$

- $\tilde{u}_\theta^{(s+1)}$ serves as a coarse initialization at the finer resolution.
- Then, we learn a residual correction $r^{(s+1)}$:

$$u_\theta^{(s+1)} = \tilde{u}_\theta^{(s+1)} + r^{(s+1)}$$

What is Upsampling?

- **Upsampling** increases the spatial resolution of a tensor (e.g., from 16×16 to 32×32).
- Common methods:
  - Nearest Neighbor: simple copy, not smooth.
  - Bilinear Interpolation: smooth, differentiable.
  - Learned Upsampling: via transposed convolutions.
- In MSFM, we use **bilinear interpolation**:
  - Efficient and parameter-free.
  - Produces smooth flow fields.
  - Fully differentiable — suitable for training.

Computational Cost

- **Per-step cost** depends on the type of solver:
  - **Explicit solvers** (e.g., Euler): fast but less stable.
  - **Implicit or adaptive solvers**: more stable, but require solving nonlinear systems $\rightarrow$ slower per step.
- **Stochastic solvers** (SDE-based) often need many steps for accurate sampling.
- Trade-off: computation time vs numerical stability and generation quality.

GPU Memory Trade-offs

- **Naive backpropagation**: stores all intermediate states $\rightarrow$ high memory usage.
- **Adjoint method** (used in CNFs):
  - Recomputes trajectory during backward pass.
  - **Reduces memory**, but **increases compute time**.
- Memory–compute trade-off is critical when scaling to high dimensions or long time horizons.
- Solvers with adaptive time steps also have varying memory needs.

Hyperparameter Tuning

- **Learning Rate**
  Controls the size of updates. Too high: unstable training. Too low: slow convergence. *Schedulers* (step decay) adjust it during training to improve stability.

- **Weight Decay**
  Regularization term that penalizes large weights. Helps reduce overfitting and improves generalization.

- **Interpolation Schedule**
  Governs how $t \in [0, 1]$ is sampled during training.
  - *Linear*: uniform sampling.
  - *Custom*: emphasizes complex regions.

  Impacts both training efficiency and sample quality.

Why Use a Non-Uniform Interpolation Schedule?

- If flow complexity evolved linearly over time, uniform sampling ($t \sim \mathcal{U}(0,1)$) would be fine.
- In practice (e.g., MNIST generation):
  - Early stages ($t \approx 0$): noise-like, easy to model.
  - Late stages ($t \approx 1$): structured images, harder dynamics.
- **Solution:** Sample more t near 1 using schedules like:

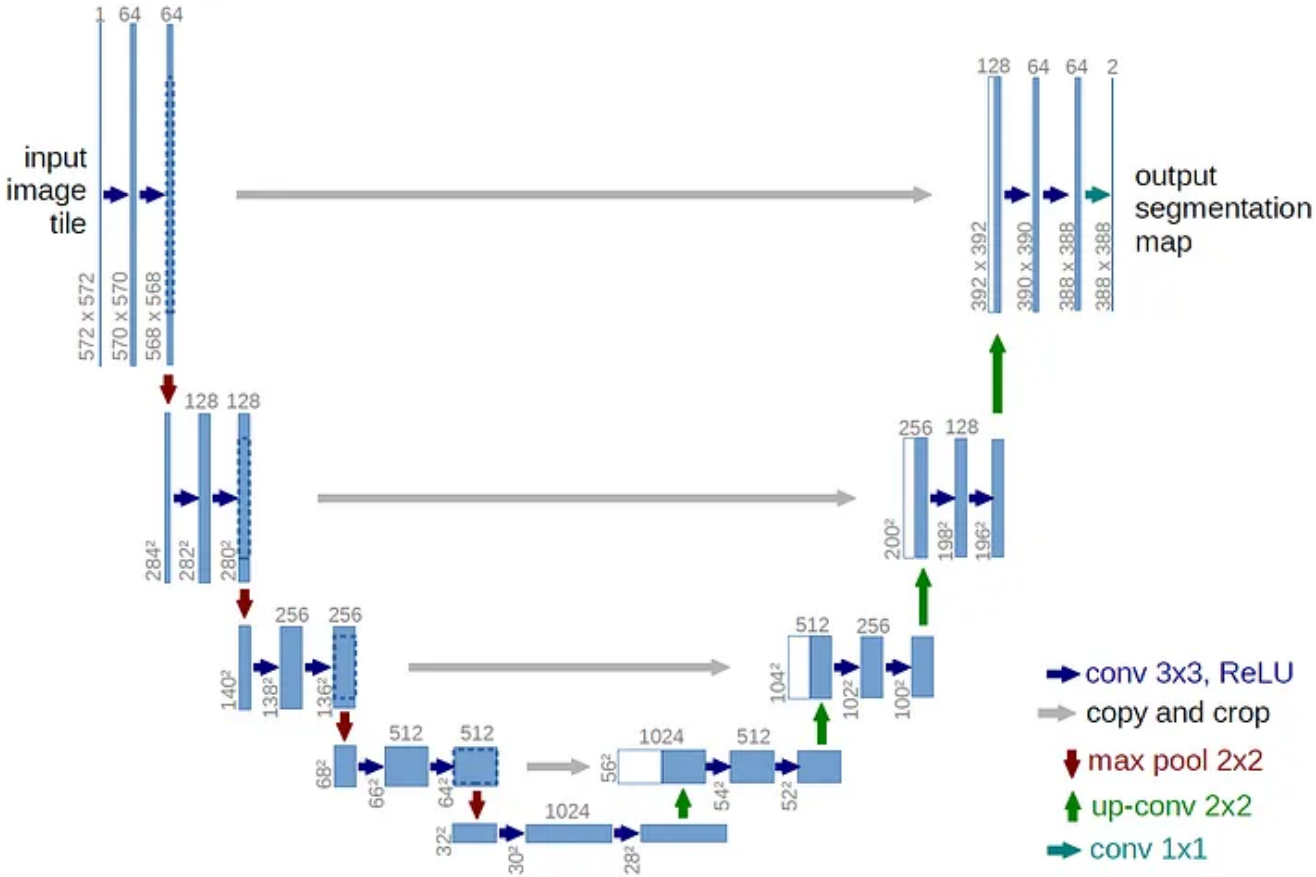
$$t \sim \text{Beta}(5,1)$$

- Focuses learning where it matters most $\rightarrow$ better generation.
- More details on the Beta distribution: `wikipedia.org/wiki/Beta_distribution`

Network Architecture Choices: MLP vs U-Net for Images

- **MLP (Multilayer Perceptron):**
  - Consists of fully connected layers without explicit spatial structure modeling.
  - Suitable for low-dimensional or synthetic datasets.
  - Limited for images because it does not capture local pixel correlations effectively.
- **U-Net:**
  - Convolutional encoder-decoder architecture with skip connections.
  - Preserves fine spatial details while capturing global context.
  - Can be extended with attention blocks to model long-range dependencies.
- **Our case study:**
  - For MNIST, the simplicity and low complexity allow effective use of an MLP.
  - For CIFAR-10, higher visual complexity requires a U-Net architecture to achieve good performance.

U-Net Architecture



Understanding U-Net Structure

- **Encoder:** Applies convolutional layers followed by downsampling (max pooling) to extract hierarchical features. Reduces spatial resolution while increasing channel depth.
- **Decoder:** Uses upsampling operations (transposed convolutions) to gradually reconstruct spatial dimensions.
- **Skip connections:** Directly connect encoder and decoder layers at the same resolution to preserve fine-grained spatial information.
- **Why it works ?** Enables the model to leverage both high-level abstract representations and low-level details which is not possible with an MLP, which flattens the input and loses all spatial structure.

Network Architecture Choices: Time Embedding Strategies

- **Fourier embeddings:** sinusoidal functions capturing multiple frequencies, widely used and effective.
- **Learned MLP embeddings:** nonlinear mappings, more flexible but require more training and less interpretable.
- Fourier embeddings are robust and sufficient in most cases; MLP embeddings can help if time conditioning is complex.

Why do we need time embeddings?

- It might seem natural to simply concatenate the scalar time t to the input.
- However, this forces the network to learn temporal progression solely from a single raw value.
- In practice, this often leads to poor performance and unstable training.
- Like positional encodings in Transformers, rich time embeddings provide useful structure.

Two main strategies are widely used:

- Fixed Fourier embeddings using sinusoidal functions.
- Learned embeddings via small MLPs.

Fourier Time Embeddings

- **Principle:** encode the time scalar $t \in [0, 1]$ using sinusoidal functions at multiple frequencies:

$$\gamma(t) = \left[\sin(2^0 \pi t), \cos(2^0 \pi t), \ldots, \sin(2^{L-1} \pi t), \cos(2^{L-1} \pi t)\right]$$

- **Advantages:**
  - Multi-scale representation of temporal variations.
  - Non-learned, thus stable and simple to implement.
  - Effective for modeling periodic or continuous dynamics.
- **Limitations:**
  - Fixed representation, less flexible.
  - Does not adapt to specific data characteristics or tasks.

MLP Time Embeddings

- **Principle:** learn a time embedding by passing t through a small dense neural network (MLP):

$$t_{\text{embed}} = \text{MLP}(t)$$

- **Advantages:**
  - Flexible and more expressive.
  - Can model complex temporal dynamics.
  - Adapted to data and task specifics.
- **Limitations:**
  - Requires more data and training time.
  - Less stable, prone to overfitting.
  - Less interpretable.

Sampling in a Flow Matching Algorithm

- Sampling occurs during the generation of new data.
- Starting from noise, we use the learned flow to extend the trajectory.
- This trajectory is described by an ordinary differential equation (ODE) of the form:

$$\frac{d}{dt}\psi_t = u_t^\theta(\psi_t(x))$$

where u_t^θ is the learned vector field.

Numerical Integration Problem

- To generate a sample, we need to numerically integrate this ODE from $t = 0$ to $t = 1$.
- Choosing the integration method (solver) is crucial to balance:
  - The **quality** of generated samples.
  - The **computational cost**, which should be as low as possible.
- Two main families of solvers:
  - **Fixed-step Euler integration**
  - **Adaptive solvers** (RK45)

Fixed-step Euler Integration — reviewed earlier on slide 10

- Simple and fast method.
- Update at each fixed step Δt using:

$$x_{k+1} = x_k + \Delta t \cdot u_{t_k}(x_k).$$

- Advantages:
  - Easy to implement.
  - Low computational cost.
- Disadvantages:
  - Limited accuracy, especially if Δt is large.
  - May require very small step sizes, increasing computation time.
- Suitable when speed is prioritized.

Adaptive Solvers (RK45) — Concept

- Adaptive solvers dynamically adjust the step size during integration to control local error.
- RK45 is a popular adaptive Runge-Kutta method that computes two solutions of different orders (4 and 5).
- By comparing these two solutions, it estimates the local error.
- If the error is too large, the step size is reduced; if small, the step size can be increased.
- This approach balances accuracy and computational cost.

- **For more details and equations, see:**
  `https://en.wikipedia.org/wiki/RungeKuttaFehlberg_method`

Research Directions

- **Integration with energy-based models:** Combining flow matching with energy-based models could enhance sample quality and model robustness by leveraging the strengths of both approaches. Energy-based models provide flexible likelihood estimation and better control over data distributions, which can complement the efficiency of flow-based generative methods.
- **Learned interpolation schedules:** Instead of fixed or hand-crafted schedules for interpolation between distributions during training or sampling, learning these schedules dynamically can improve convergence speed, stability, and sample diversity. Adaptive schedules tailor the training process to the data and model behavior, potentially leading to better performance.

Applications Beyond Imaging

- **Molecular design:** Generative models help create new molecules with desired properties for drug discovery and materials science, accelerating the search for novel compounds.
- **Speech synthesis:** Flow matching and diffusion-based models improve naturalness and diversity of synthesized speech, enabling realistic voice assistants and audio content generation.
- **Physics simulation:** Continuous generative models assist in approximating complex physical systems, enabling faster simulations in fluid dynamics, climate modeling, and quantum mechanics.
- **Control:** Generative frameworks facilitate learning control policies in robotics and autonomous systems, allowing flexible adaptation to uncertain or dynamic environments.

Conclusion

- Flow Matching unifies continuous generative modeling with efficient training and sampling.
- Stochastic variants improve stability and sample diversity.
- Key architectures like U-Net and numerical integration methods (e.g., Runge-Kutta) critically impact model accuracy and speed.
- Advances in image generation have enabled high-quality, controllable, and diverse content creation across domains such as art, design, and scientific visualization.